Firewall Implementation — Project Report

Computer Networks course project

Name: Deepesh Kumar Kotta

USN: 1CR24AI035

Sem & Section: 5-A

Live demo: https://pywall-demo.onrender.com/ (read-only, simulated traffic) · **Source code:** https://github.com/DE9856/Firewall-Implementation

Abstract

This project builds **PyWall**, a stateful packet-filtering firewall for Linux written in Python, together with a web dashboard that shows how it treats every packet. PyWall receives each IPv4 packet entering or leaving the host from the kernel through netfilter's NFQUEUE interface, parses the headers itself from raw bytes, and returns a verdict: accept or drop. The decision passes through a fixed pipeline: header parsing, white and black lists, intrusion-detection bans, intrusion detection, connection tracking, an ordered rule table, and a default policy.

PyWall detects SYN floods, port scans and ping floods and bans the source. It drops spoofed TCP packets and unsolicited replies, and it rate-limits selected traffic. The dashboard draws the traffic as streams flowing through the pipeline and explains each verdict field by field. It also lets the user edit rules while the firewall runs.

The firewall was tested in four ways:

- **Simulation:** 73 of 73 simulated packets got the expected verdict.
- **Unit tests:** all 43 pass.
- **Private test network:** a recorded live run on a virtual network blocked a real port scan and a real SYN flood.
- **Real Wi-Fi:** on a home network, outbound Telnet was blocked while web browsing, DNS and ping kept working, including connections that were already open when the firewall started.

Firewalls in plain language

How data travels. When your laptop loads a web page, the data does not travel as one piece. It is cut into thousands of small **packets**, like letters in the post. Each packet carries a label on the outside, its **header**, with:

- **who sent it:** the source address, e.g. 192.168.1.26

- **who it is for:** the destination address
- **which service it is meant for:** the **port**. Port 443 is secure web pages, 22 is remote login (SSH), 53 is name lookups (DNS).
- **what kind of conversation it belongs to:** the **protocol**, such as TCP for web pages

The contents inside, such as the text of the web page, are the **payload**.

What a firewall does. A firewall is like the security desk in the lobby of an office building. Every visitor (packet) arriving or leaving must pass the desk. The guard does not open the visitors' bags (the payload); they only read the badge (the header) and compare it with a **list of rules** on the desk, such as:

- "anyone may visit the public reception" (anyone may reach the web server on port 80)
- "only staff may enter the server room" (only computers on our own network may log in on port 22)
- "nobody takes the old service lift" (no outgoing Telnet, an insecure protocol)

The guard reads the list from the top, and the **first rule that fits** decides. If no rule fits, the visitor is turned away. This is called **default deny**: whatever is not explicitly allowed is blocked. Each decision is a **verdict**: **ACCEPT** (let it through) or **DROP** (stop it silently).

Why it remembers conversations. When you visit a website, the website's reply is a stranger arriving at the desk. A simple guard would turn it away, because no rule says "let in random web servers". A smarter guard keeps a visitor book: "our employee just went out to meet example.com, so example.com's reply is expected". That visitor book is **connection tracking**, and a firewall that keeps one is **stateful**. It also catches fakes: a stranger who claims to be continuing a conversation that never started is stopped.

Why it watches behaviour. Some attacks are not one bad packet but many in a hurry:

- **Port scan:** someone rattles every door handle in the building to see which is unlocked, by trying many ports quickly.
- **SYN flood:** someone sends crowds of fake visitors to jam the entrance, with many connection requests at once.

PyWall's **intrusion detection** counts what each address does per second. When an address crosses a limit, PyWall bans it for 60 seconds, like a guard who stops letting a troublemaker in at all.

What PyWall is. PyWall is such a guard, written in Python for Linux. The Linux kernel passes it every packet and waits for its verdict before delivering or discarding the packet. The rest of this report explains how it is built, how it was tested, and how its dashboard lets anyone watch the guard at work.

1. Introduction and objectives

A firewall sits between a host or network and the outside world. It allows or denies traffic according to a security policy. Without one, every open service on a machine can be reached from any address, which exposes it to scanning, brute-force logins and denial-of-service attacks.

The objectives of this project were:

1. Capture real packets from the operating system and parse their IPv4, TCP, UDP and ICMP headers.
2. Filter packets with a configurable, ordered rule table and a default-deny policy.
3. Track connections, so that replies to allowed traffic pass and stray packets are dropped.
4. Detect common attacks (SYN flood, port scan, ICMP flood) and ban the attacker automatically.
5. Rate-limit selected traffic, such as ping.
6. Log every verdict, report statistics, and test the firewall without root access.
7. Make the firewall's behaviour visible and explainable in a real-time web dashboard, including rule editing.
8. Run safely on a real network for a live demonstration, without cutting connections that are already open.

1.1 Key terms

Term	Meaning in this project
Packet	One unit of data on an IP network: headers (addresses, ports, flags) followed by a payload. PyWall reads only the headers.
IPv4 header	The first 20+ bytes of a packet. It holds the source and destination address, the protocol number (1 ICMP, 6 TCP, 17 UDP), the TTL and the total length.
Port	A 16-bit number that identifies a service on a host, such as 22 for SSH, 53 for DNS or 443 for HTTPS.
TCP flags	Bits in the TCP header that mark the stage of a connection: SYN opens it, ACK acknowledges, FIN closes, RST aborts.

Term	Meaning in this project
ICMP	The control protocol behind ping (type 8 echo request, type 0 echo reply) and network error messages (types 3 and 11).
CIDR	A way to write a range of addresses, e.g. 192.168.1.0/24 means the 256 addresses 192.168.1.0 to 192.168.1.255.
Firewall	Software that allows or blocks traffic according to a security policy.
Packet filter	A firewall that decides from packet headers (addresses, ports, protocol, flags).
Stateful firewall	A packet filter that also remembers connections, so replies to allowed traffic pass automatically and stray packets are dropped.
Rule / rule table	A condition and an action (ACCEPT or DROP). Rules are checked from the top and the first match decides.
Default policy	The verdict for traffic no rule matches. PyWall uses DROP: deny by default.
Verdict	The firewall's decision for one packet. PyWall has two: ACCEPT (allowed, forwarded) and DROP (held, discarded).
5-tuple	Protocol, source address, source port, destination address and destination port. Together they identify one connection.
Connection tracking (conntrack)	The table of known connections. A packet is NEW (may open a connection), ESTABLISHED (belongs to one) or INVALID (belongs to none and cannot open one).
Fast path	ESTABLISHED packets are accepted at the connection-tracking stage without checking the rule table.
Connection pickup	Adopting a connection that was already open when the firewall started, from its outbound side. This is the same idea as the Linux kernel's loose TCP pickup.

Term	Meaning in this project
IDS	Intrusion detection system: here, detectors for SYN floods, port scans and ping floods that ban the offending source for 60 seconds.
SYN flood	Many connection requests (SYN packets) in a short time, meant to exhaust the target.
Port scan	Probing many ports of a host to find open services, typically the first step of an attack.
Rate limit / token bucket	Allowing at most N packets per period per source. The bucket refills over time, so short bursts are fine but a sustained rate is capped.
Spoofed packet	A packet with forged header fields, e.g. a TCP ACK that claims to belong to a connection that never existed.
Netfilter	The packet-filtering framework inside the Linux kernel (configured with iptables or nftables).
NFQUEUE	A netfilter target that hands packets to a user-space program and waits for its verdict. This is how PyWall filters real traffic.
pcap	The standard capture file format written by tcpdump and opened by Wireshark. PyWall can replay one through its engine.
veth / network namespace	A virtual network cable and an isolated network stack. Together they make a private test network on one machine.
Dry run	Live mode that computes and shows every verdict but drops nothing, for safe rehearsal.
REST API / SSE	The dashboard reads snapshots over HTTP (REST) and receives live updates over Server-Sent Events, a one-way stream from server to browser.

2. Background

PyWall is a stateful packet filter. It works at the network and transport layers (OSI layers 3 and 4). The table compares the main kinds of firewall.

Type	OSI layers	Decides on	Strength	Weakness
Stateless packet filter	3–4	Each packet's header alone (IPs, ports, protocol)	Fast, simple	Cannot tell a reply from a forged packet
Stateful packet filter (this project)	3–4	Header plus the state of the connection it belongs to	Accepts replies automatically, drops spoofed packets	Does not inspect payloads
Application proxy / gateway	7	Content of the application protocol (HTTP, FTP)	Deep inspection	Slower, one proxy per protocol
Next-generation firewall	3–7	Headers, state, applications, users, signatures	Most complete	Complex and expensive

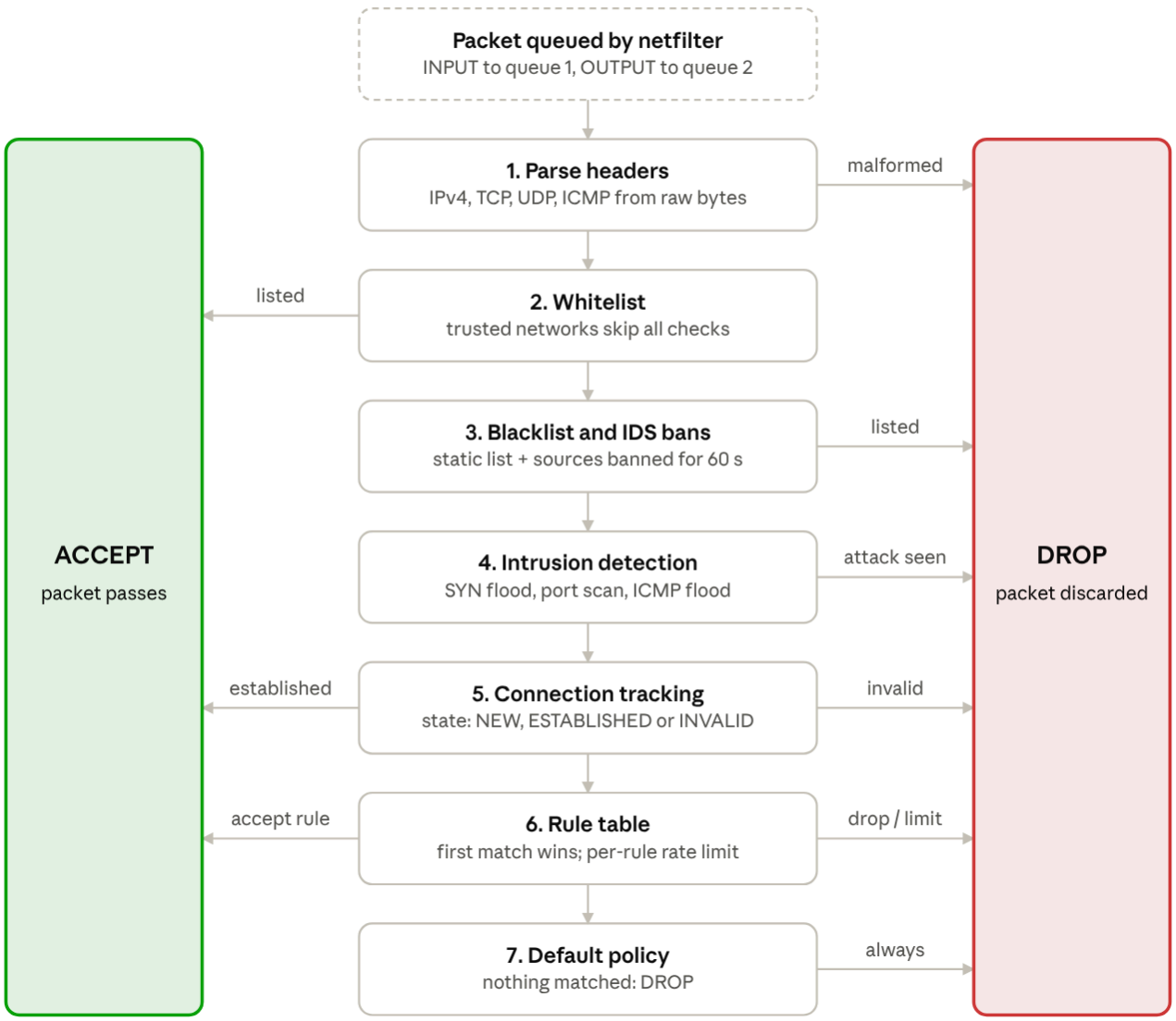
Headers used. The IPv4 header supplies the source and destination address, the protocol number (1 = ICMP, 6 = TCP, 17 = UDP), the TTL and the length. TCP and UDP supply the port numbers. TCP also supplies the flags SYN, ACK, FIN and RST, which drive connection tracking. ICMP supplies the message type (8 = echo request, 0 = echo reply, 3 = unreachable, 11 = time exceeded).

TCP three-way handshake. A connection opens with SYN → SYN-ACK → ACK and closes with FIN or RST. Any packet without SYN only makes sense inside an existing connection. A stateful firewall uses this to reject forged packets.

Netfilter and NFQUEUE. Linux filters packets in the kernel with *netfilter*, configured through iptables or nftables. The NFQUEUE target suspends a packet and hands it to a user-space program. The program returns a verdict (accept or drop), and the kernel applies it. PyWall uses this mechanism, so its decisions are enforced on real traffic.

3. System design and architecture

Every packet runs through one fixed pipeline. The first stage that can decide returns the verdict, so cheap checks and ban lists run before the rule table. The kernel holds each packet until the verdict arrives, then delivers or discards it.



PyWall decision pipeline · 7 stages, 2 verdicts

When a NEW packet is accepted, its flow is added to the connection table. Its replies then exit at stage 5 without reaching the rules. Intrusion detection inspects inbound traffic only.

The same engine accepts packets from three sources:

- **Live:** packets from the kernel through NFQUEUE, with verdicts enforced.
- **pcap:** a tcpdump capture replayed offline.
- **Simulated:** attack and normal traffic generated in code.

All three call `Firewall.process(raw_bytes, direction)`, so a result from testing carries over to live use. Each source can run from the command line on its own, or behind the web dashboard with `pywall.py serve`.

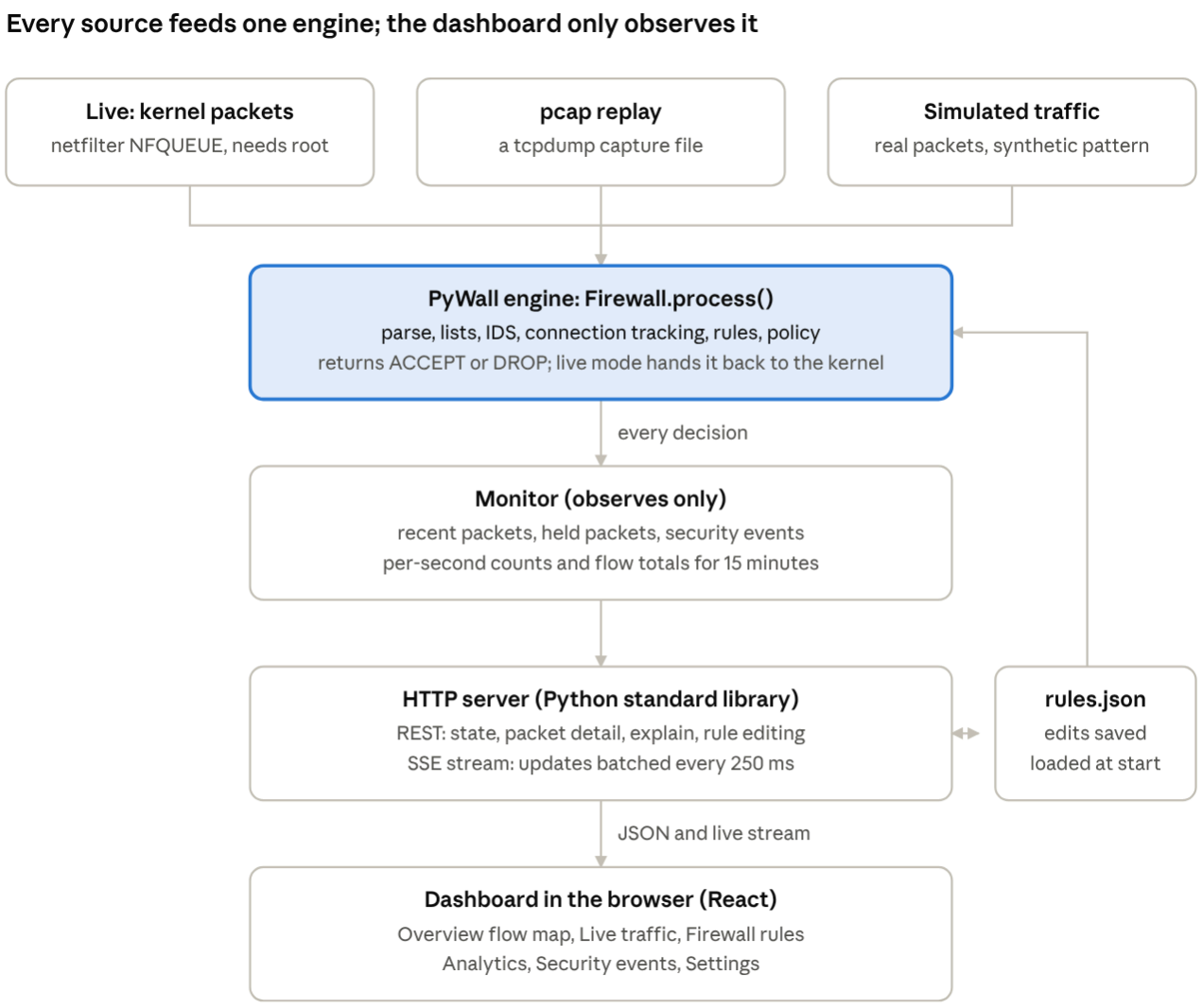
The policy lives in one JSON file, `config/rules.json`. It holds the default policy, the local networks, the black and white lists, the IDS thresholds and the ordered rules:

Order	ID	Action	Direction	Match	Purpose
1	1	ACCEPT	any	src 127.0.0.0/8	Loopback
2	2	DROP	out	tcp dport 23	Block insecure Telnet
3	3	ACCEPT	out	anything	This host may open outbound connections
4	11	ACCEPT	in	udp sport 67, dport 68	DHCP replies, so the network lease renews
5	4	ACCEPT	in	tcp dport 22, src $LAN	SSH from the local network only
6	5	DROP	in	tcp dport 22	SSH from anywhere else
7	6	ACCEPT	in	tcp dport 80, 443	Public web server
8	7	DROP	in	tcp dport 445, 3389, 135–139	SMB, RDP, NetBIOS
9	8	ACCEPT	in	icmp type 8, 5 per second	Ping, rate limited
10–11	9–10	ACCEPT	in	icmp type 3, 11	Unreachable and time-exceeded errors
—	—	DROP	—	default policy	Everything else

`$LAN` stands for the local networks: 192.168.1.0/24 by default, or the subnet detected on the demo machine with `--lan auto`.

3.1 How the parts fit together

The dashboard was added without changing how verdicts are made. It hangs off the engine as a listener that only observes.



PyWall architecture · 3 sources, 1 engine, 1 observer

The monitor and server run in background threads and share one lock with the engine, so a rule edit is swapped in between two packets and never in the middle of a decision. The browser loads one snapshot over REST, then follows a Server-Sent Events stream. Updates are batched every 250 ms, so a SYN flood does not flood the browser too.

3.2 Following one packet

Two examples show the pipeline from the diagram above at work, using the rule set in this section. The laptop's address is 192.168.1.26.

Example 1: you open a website (allowed).

1. Your browser sends the first packet of a new connection: from 192.168.1.26, port 51000, to example.com (104.20.23.154), port 443, with the TCP flag SYN ("let's start talking").
2. **Parse:** the header is valid IPv4 and TCP.

3. **Lists:** the address is on neither the whitelist nor the blacklist, so the packet moves on.
4. **Bans and IDS:** nobody is banned, and an outgoing packet is not counted by the attack detectors.
5. **Connection tracking:** no conversation with example.com is known yet, but a SYN may start one, so the packet is NEW and goes to the rule table.
6. **Rules:** rule 2 ("block outbound Telnet") asks for port 23 and does not match. Rule 3 ("allow all other outbound") matches. The verdict is **ACCEPT**, and the conversation is written into the connection table.
7. **The reply:** example.com answers from port 443 to port 51000. Connection tracking finds the conversation in its table and accepts the reply on the spot, on the **fast path**, without checking a single rule. Every later packet of this page load takes the same shortcut.

Example 2: a stranger tries to log in to your laptop (blocked).

1. A computer on the internet, 45.33.32.156, sends a SYN to your laptop's port 22 (SSH, remote login).
2. **Parse, lists, bans:** the packet is valid, the address is not listed, and it is not banned.
3. **IDS:** this is the stranger's first attempt, far below the port-scan and SYN-flood limits, so the detectors only count it.
4. **Connection tracking:** a SYN may start a conversation, so the packet is NEW.
5. **Rules:**
    - Rules 1 to 3 do not fit: wrong source, or outgoing only.
    - Rule 11 (DHCP) asks for UDP and does not fit.
    - Rule 4 ("SSH only from the LAN") asks for a source inside 192.168.1.0/24. The packet comes from 45.33.32.156, so this condition fails.
    - Rule 5 ("drop SSH from elsewhere") matches: inbound, TCP, port 22.
6. The verdict is **DROP**. The packet is discarded without any reply, so the stranger cannot even tell whether an SSH service exists.

If the same stranger tried 16 different ports within 5 seconds, the IDS would decide earlier, at its own stage. It would ban the address for 60 seconds, and every further packet from it would be dropped at the **Bans** stage without reaching the rules.

The dashboard shows exactly these explanations: click any packet in **Live traffic** and the inspector lists every stage it passed, the rule that matched, and each of that rule's conditions.

4. Implementation

PyWall is about 2,300 lines of Python 3 (plus 420 lines of tests), and the dashboard about 3,200 lines of TypeScript and CSS. The firewall and its server use only Python's standard library. The one exception is `NetfilterQueue`, which binds to the kernel's packet queue in live mode.

Module	Responsibility
`pywall.py`	Command-line interface: `check`, `simulate`, `pcap`, `live`, `serve` (dashboard), `doctor` (pre-flight check).
`firewall/packet.py`	Parses IPv4, TCP, UDP and ICMP headers with `struct`, and builds test packets with correct RFC 1071 checksums. Also dissects headers field by field, with byte offsets, for the packet inspector.
`firewall/rules.py`	Rule model, CIDR/port/flag matching, the `$LAN` token, JSON loading and validation, the token bucket, and per-condition checks for explanations.
`firewall/conntrack.py`	Connection table with NEW / ESTABLISHED / INVALID states, timeouts, and mid-stream pickup.
`firewall/ids.py`	Sliding-window detectors for SYN flood, port scan and ICMP flood, and the temporary ban list.
`firewall/engine.py`	The decision pipeline. Each decision records its stage, direction, time and header bytes, and is passed to listeners.
`firewall/explain.py`	Read-only replay of the pipeline for one packet. It reports what each stage would conclude without changing any counter or table.
`firewall/live.py`	Installs the iptables NFQUEUE hooks, returns verdicts to the kernel, and removes the hooks on exit.
`firewall/netinfo.py`	Finds the network interface that carries the default route and its subnet (for `-i auto` and `--lan auto`).
`firewall/monitor.py`	Dashboard data: recent and held packets, coalesced security events, per-second counts, and flow totals.

Module	Responsibility
`firewall/server.py`	Standard-library HTTP server: REST API, Server-Sent Events stream, validated rule editing, static files.
`firewall/traffic.py`	The simulated traffic source: normal sessions plus recurring attack episodes, built from real packets.
`firewall/pcap.py`	Reads tcpdump captures (Ethernet, raw, Linux cooked) for offline replay.
`firewall/simulator.py`	The 11 scripted test scenarios with a virtual clock.
`web/`	Dashboard source (React, TypeScript, Vite). The built files are committed in `firewall/web/`, so running it needs no Node.js.
`demo/`	The private test network (`live_demo.py`), attack probes, the live checker (`check_live.py`), and the demo videos.

4.1 Packet parsing

The kernel delivers each packet starting at the IPv4 header. The first 20 bytes are unpacked in network byte order. The IHL field gives the header length, which locates the transport header.

```
ver_ihl, _tos, total_len, _id, _frag, ttl, proto, _csum, src, dst = \
    struct.unpack("!BBHHHBBH4s4s", data[:20])
ihl = (ver_ihl & 0x0F) * 4          # header length in bytes
payload = data[ihl:]
if proto == PROTO_TCP:
    src_port, dst_port = struct.unpack("!HH", payload[:4])
    tcp_flags = payload[13]          # FIN=0x01 SYN=0x02 RST=0x04 ACK=0x10
```

Truncated packets, packets with a bad header length and non-IPv4 packets raise `PacketError`. The engine drops them.

4.2 Rule matching

Rules are JSON objects and are checked top to bottom. Every field a rule specifies must match, and fields it leaves out match anything. The first matching rule decides. When no rule matches, the default policy (DROP) applies.

```
{"name": "SSH only from the LAN", "action": "ACCEPT", "direction": "in",
 "protocol": "tcp", "dst_port": 22, "src": "192.168.1.0/24", "log": true}
```

Supported fields are `action`, `direction` (in/out/any), `protocol`, `src` and `dst` (CIDR or list), `src_port` and `dst_port` (a number, `"6000-6100"` or a list), `icmp_type`, `tcp_flags` (e.g. `"S!A"` = SYN set, ACK clear), `rate_limit` (e.g. `"5/s"`) and `log`. Invalid rules are rejected when the file loads, not when traffic arrives.

4.3 Connection tracking

Each flow is keyed by the 5-tuple (protocol, source IP, source port, destination IP, destination port). A packet is looked up under its own key and under the reversed key, so replies are found.

- **ESTABLISHED**: the flow exists, so the packet is accepted without consulting the rules.
- **NEW**: the packet may open a flow (TCP SYN without ACK, any UDP datagram, an ICMP echo request). It is judged by the rules and recorded if accepted.
- **INVALID**: the packet has no flow and cannot open one, such as a lone ACK or an unsolicited echo reply. It is dropped.

TCP flows move from SYN_SENT to ESTABLISHED on SYN-ACK. They switch to a 10-second timeout on FIN and are deleted on RST. Idle flows expire after 30 s (half-open), 3600 s (TCP), 60 s (UDP) or 30 s (ICMP).

4.4 Intrusion detection and rate limiting

For every inbound packet, the detector records a timestamp per source address in a sliding window.

Attack	Trigger (default)	Response
SYN flood	More than 20 SYNs from one source in 1 s	Ban source for 60 s
Port scan	More than 15 distinct destination ports from one source in 5 s	Ban source for 60 s
ICMP flood	More than 10 echo requests from one source in 1 s	Ban source for 60 s

Rate limits use a token bucket per rule and per source. A bucket holds N tokens and refills at N per period. Each accepted packet spends one token, and a packet that finds the bucket empty is dropped. The bucket allows short bursts but caps the sustained rate.

4.5 Running on a real network: connection pickup

When a firewall starts on a machine that is already online, connections are already open, such as browser tabs or an SSH session. A strict connection tracker has never seen them open, so it would drop their packets as INVALID and cut them. PyWall therefore adopts such a connection from its outbound side, in the same spirit as the Linux kernel's loose TCP pickup:

1. An outbound TCP packet (not RST) with no known connection is judged by the rule table as if it were NEW.
2. If a rule allows it, the connection is recorded straight away as ESTABLISHED.
3. Its replies then take the fast path.

Inbound packets with no connection are still INVALID and dropped, so spoofed or unsolicited traffic gets no new opening. The behaviour can be switched off with `"pickup_outbound": false`.

Rules can name the local network as `$LAN` instead of a fixed range. With `--lan auto`, PyWall reads the subnet of the interface that carries the default route, and every `$LAN` rule follows it. The same rule set therefore works on any demo network.

4.6 Explaining a verdict

To explain a verdict, `explain()` walks a copy of the pipeline. It reports, stage by stage, what each one concludes, and for each rule it lists every condition with the rule's value, the packet's value and whether it matched. It changes nothing: no hit counters, no rate-limit tokens, no IDS counters, no connection entries. A unit test checks that its verdict and deciding stage always equal those of the real engine.

4.7 The monitoring dashboard

The monitor subscribes to every decision and keeps, for the session:

- the latest 5,000 packets, plus the latest 1,000 dropped packets separately, because drops are rare and worth inspecting
- per-second counters for 15 minutes
- flow totals by source, destination, service, stage and verdict
- security events: repeats of one kind from one source within 10 seconds are grouped into a single event with a count

The server sends one snapshot, then a batch of new data every 250 ms over Server-Sent Events. Requests that change rules must carry an `X-PyWall: 1` header and a JSON body. A web page from another site cannot send that header without permission the server never grants, which blocks cross-site attacks. The server binds to 127.0.0.1 only.

Rule edits are checked by the same validator the firewall uses at start-up. They are swapped into the running rule table under the engine lock, saved atomically to `config/rules.json` with the file's original owner and permissions kept, and recorded as configuration events.

The front end is a single-page React application with six sections:

- **Overview:** a flow map. Sources on the left feed a frame with one column per pipeline stage; dropped traffic turns down into the bin of the stage that stopped it, and forwarded traffic continues to its service on the right. Stream width is the packet count on one shared scale, and each moving dot is one real packet, sampled when traffic is heavy.
- **Live traffic:** a Wireshark-style packet list with filters, and an inspector that shows the trace, the matched rule's conditions, every header field and the raw bytes.
- **Firewall rules:** the rule table in plain language, with editing and a packet tester.
- **Analytics:** volume over time, direction, protocols, top sources and destinations, rules matched, and where packets were dropped.
- **Security events:** a timeline of events.
- **Settings:** the connection and the configuration.

Colour carries meaning only: teal for allowed, coral with a 45-degree hatch for dropped, amber for IDS and rate limits. Each is paired with a shape (ring, hatch, diamond), so the screen stays readable for colour-blind viewers. Motion follows the user's reduced-motion setting.

The Overview is also designed for people who have never seen a firewall:

- **First-visit guide:** a short panel explains what PyWall is and what the colours and dots mean. It can be closed and brought back.
- **Plain-language labels:** the three parts of the map are named (computers sending traffic, PyWall's checks, where allowed traffic goes), and every stage name explains itself on hover.
- **Slow motion:** a speed control slows the animation down to one tenth of normal. Traffic is still decided in real time; only the drawing slows.
- **Blocked only:** shows just the blocked packets, each tagged with why it is being stopped, such as "rule 7: block SMB" or "banned source".
- **Packet journey:** follows one real blocked packet slowly through the pipeline. The packet pauses at every stage it passed, while a side panel says what each stage found, up to the stage that stopped it. The panel has Pause, Next stage and Replay controls, and Back to all traffic (or Esc) returns. A journey can also be started from any packet in the Live traffic inspector.

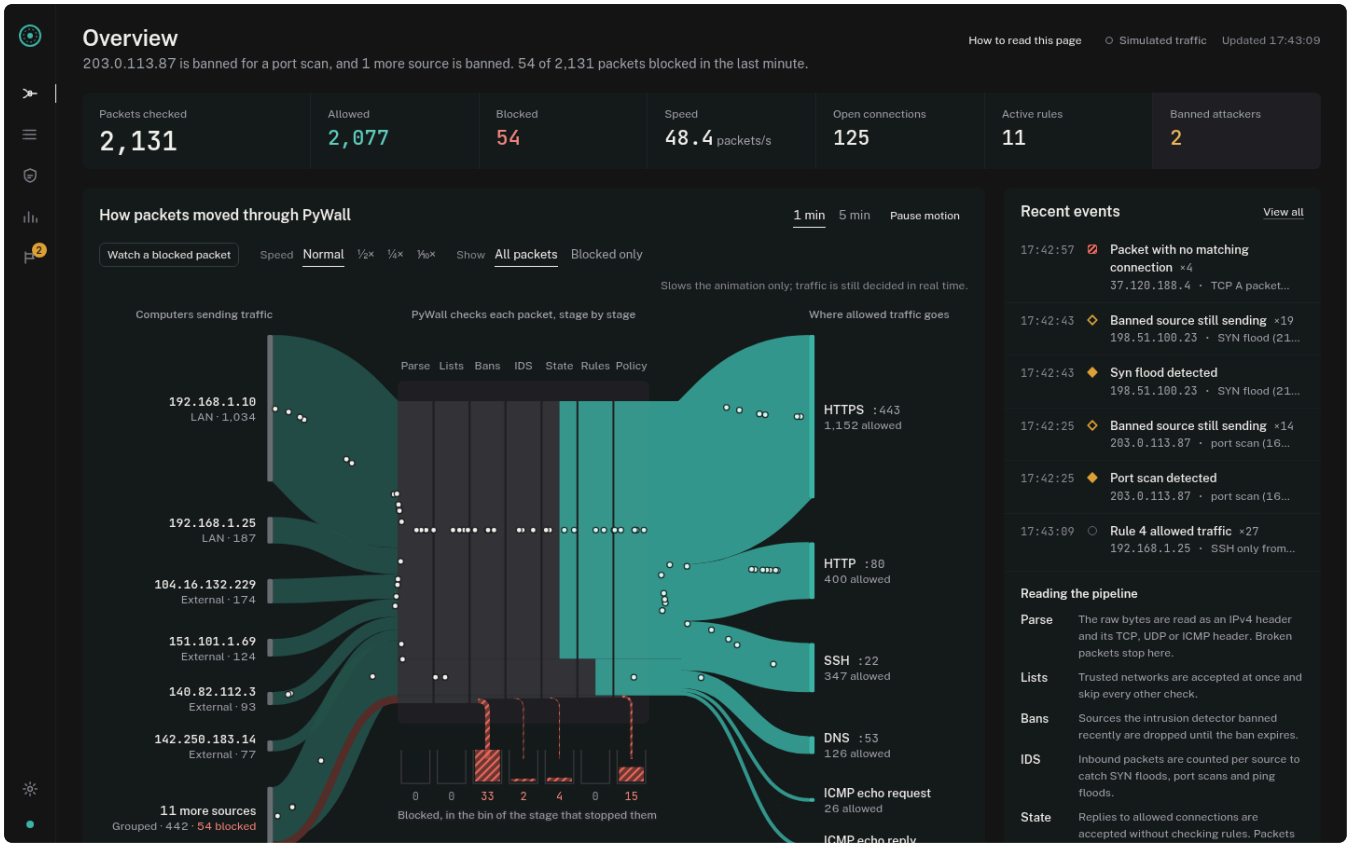


Figure 1. The Overview flow map. Stream width is the packet count; each dot is one real packet.

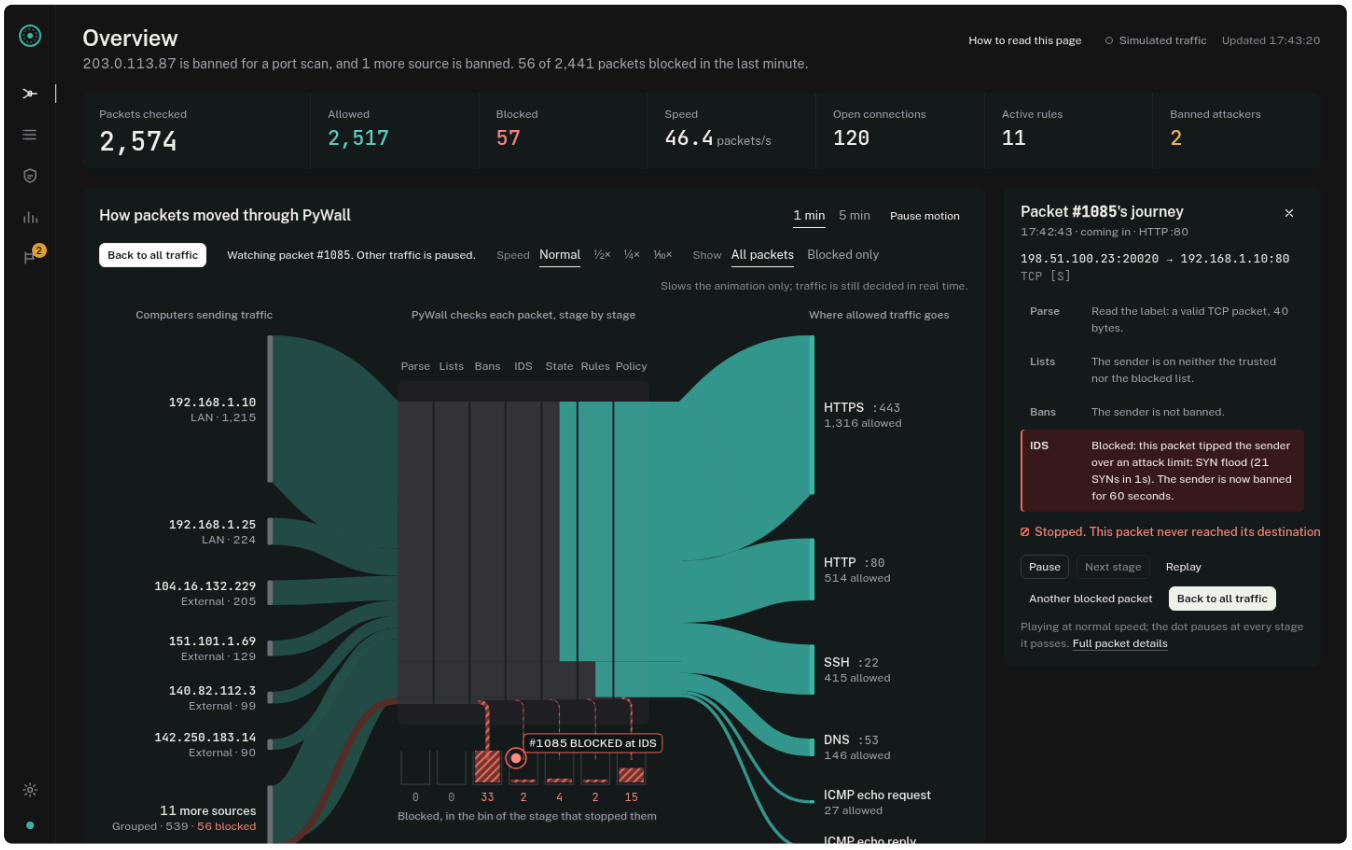


Figure 2. A packet journey. Packet 1085 from a SYN flooder passes three stages and is blocked by the intrusion detector, which then bans the sender.

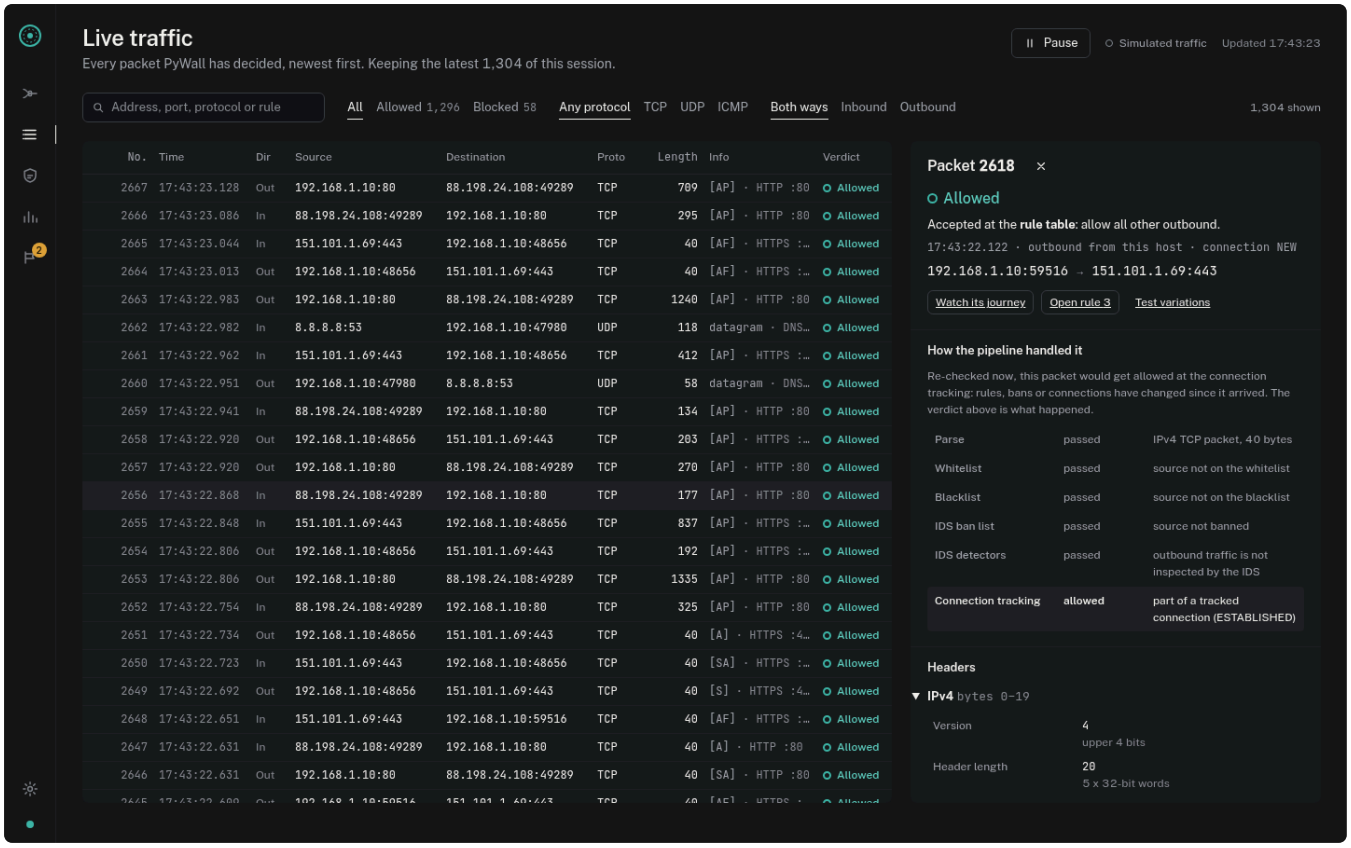


Figure 3. Live traffic and the packet inspector: the verdict, each stage's outcome, and every header field.

5. Testing and results

All 73 simulated packets got the expected verdict, and all 43 unit tests pass. The simulator builds real IPv4 packets with valid checksums and feeds them through the same engine that live mode uses. A virtual clock makes timing-based attacks repeatable. The protected host is 192.168.1.10 on the LAN 192.168.1.0/24.

#	Scenario	Packets	Expected behaviour	Result
1	Outbound HTTPS + replies	5	SYN allowed by the outbound rule; SYN-ACK, data and FIN accepted as ESTABLISHED	Pass
2	DNS query and reply	2	Query allowed out; reply accepted as ESTABLISHED	Pass
3	SSH from LAN vs internet	2	LAN accepted, internet dropped	Pass
4	Public web server (port 80)	1	Accepted	Pass

#	Scenario	Packets	Expected behaviour	Result
5	RDP, SMB, SNMP inbound; Telnet outbound	4	All dropped	Pass
6	Spoofed ACK, unsolicited echo reply and DNS reply	3	Dropped (invalid state or default policy)	Pass
7	Blacklisted source	1	Dropped even to the open port 80	Pass
8	Ping at 5/s limit	8	First 5 accepted, next 2 dropped, accepted again after 1 s	Pass
9	Port scan (20 ports in 1 s)	21	Ban on the 16th port; later port-80 SYN dropped	Pass
10	SYN flood (25 SYNs in 0.25 s)	25	First 20 accepted, 21st triggers ban, rest dropped	Pass
11	Malformed 3-byte packet	1	Dropped	Pass

Excerpt of the simulator output (`python3 pywall.py simulate`):

```
1. Outbound HTTPS + stateful reply
  ok  out TCP 192.168.1.10:51000 -> 142.250.183.14:443 [S]    ACCEPT allow
all other outbound
  ok   in TCP 142.250.183.14:443 -> 192.168.1.10:51000 [SA]   ACCEPT
established connection

6. Spoofed / unsolicited packets
  ok   in TCP 45.33.32.156:443 -> 192.168.1.10:51999 [A]      DROP   invalid
state (no matching connection)

9. Port scan detection
  ok   in TCP 203.0.113.66:60014 -> 192.168.1.10:34 [S]       DROP   default
policy
  ok   in TCP 203.0.113.66:60015 -> 192.168.1.10:35 [S]       DROP   IDS:
port scan (16 ports in 5s)
  ok   in TCP 203.0.113.66:60100 -> 192.168.1.10:80 [S]       DROP   source
banned by IDS: port scan (16 ports in 5s)

10. SYN flood detection
  ok   in TCP 198.51.100.77:30019 -> 192.168.1.10:80 [S]      ACCEPT public
web server
  ok   in TCP 198.51.100.77:30020 -> 192.168.1.10:80 [S]      DROP   IDS: SYN
flood (21 SYNs in 1s)
```

```
Simulation: 73 passed, 0 failed (73 packets)
Accepted: 35   Dropped: 38   Tracked connections: 25
IDS alerts:
  203.0.113.66: port scan (16 ports in 5s)
  198.51.100.77: SYN flood (21 SYNs in 1s)
```

The 43 unit tests (`python3 -m unittest discover -s tests -t .`) cover:

- **Packets:** parsing, checksums and header dissection.
- **Rules:** first-match order; CIDR, port-range, direction and TCP-flag matching; rejection of invalid rules; the `$LAN` token; disabled rules being skipped.
- **Rate limiting:** the token bucket.
- **Connection tracking:**
  - stateful replies for TCP, UDP and ICMP
  - closing on RST, timeout after FIN
  - adopting an already-open outbound connection while inbound strays are still dropped
- **Lists and bans:** blacklist and whitelist precedence, ban expiry, and slow legitimate traffic never being flagged.
- **Explanations:** for six kinds of packet, the read-only explanation matches the real verdict and changes no counter.
- **Monitor:** event coalescing, totals and flow sums.
- **HTTP API:**
  - snapshot and packet detail
  - the safety header requirement
  - rule create, disable, reorder and delete, each saved to the file and recorded as an event
  - invalid rules rejected with the file left unchanged
- **Capture files:** pcap reading.

5.1 Live test on real packets

In live mode, PyWall blocked every attack on real kernel traffic. Its final report counted 120 packets accepted and 122 dropped. The test used a private network: an "attacker" network namespace (10.99.0.2 and 10.99.0.3) connected to the protected host (10.99.0.1) by a veth pair, with PyWall attached through NFQUEUE to that interface only. The test ran on Fedora Linux on 10 November 2026 and is recorded in `demo/pywall_live_demo.mp4` .

Test	Firewall off	Firewall on (PyWall verdict)
Attacker fetches web page on port 80	Page returned	Page returned (rule 6 ACCEPT, replies ESTABLISHED)
Attacker connects to port 22 (SSH)	Open	Filtered, no reply (rule 5 DROP)
Attacker connects to port 3389 (RDP)	Open	Filtered, no reply (rule 7 DROP)
Host fetches web page from attacker side	—	Page returned (rule 3 ACCEPT)
Host connects out to port 23 (Telnet)	—	Filtered (rule 2 DROP)
Spoofed lone TCP ACK to port 80	—	Dropped as INVALID state
16 pings at 8 per second (limit 5/s)	2 of 2 replied	14 of 16 replied; 2 dropped by the rate limit
Port scan of ports 1–40	—	Scanner banned at the 16th port; all 40 filtered; port 80 then refused too
30-SYN flood from 10.99.0.3	—	First 20 accepted; banned on the 21st SYN; remaining packets dropped

When PyWall stopped, it removed its iptables hooks, and `iptables -S` showed no NFQUEUE rules left.

5.2 Test on a real network

On 10 November 2026, PyWall filtered a laptop's real Wi-Fi traffic (interface `wlo1`, network 192.168.1.0/24, found automatically). The script `demo/check_live.py` generated traffic and confirmed every verdict through the dashboard's API. The checks were run twice: first as a dry run (verdicts shown, nothing dropped), then with blocking on.

Check	Dry run	Blocking on
HTTPS to example.com: outbound SYN allowed by rule 3	Pass	Pass
HTTPS replies accepted on the fast path	13 of 13	12 of 12
Page fetched end to end	Pass	Pass

Check	Dry run	Blocking on
DNS query to 1.1.1.1 and its answer	Pass	Pass
Ping 1.1.1.1: 3 requests out, 3 replies accepted	Pass	Pass
Telnet (port 23) marked as dropped by rule 2	Pass	Pass
Dashboard stream delivers live updates	Pass	Pass

To prove the blocking was real, the laptop connected to portquiz.net, a public test server that accepts connections on every TCP port:

- **Ports 24 and 8080:** both connected in 0.2 s.
- **Port 23:** timed out after 5 s, and PyWall's log shows each attempt dropped by "block outbound telnet".

It was the same server; only the port differed. The Claude session used to run these checks had been open before PyWall started and kept working throughout, which confirms that connection pickup works.

The dashboard also showed the network's real background traffic: other devices' broadcasts on UDP port 57621 dropped by the default policy, and one stray inbound ACK for a connection opened before start, dropped as INVALID.

6. Step-by-step guide

6.1 Setup

Requirements:

- Linux and Python 3.10 or newer. Nothing else is needed for simulation, pcap replay, tests or the dashboard.
- For live mode only: root, iptables, and the `NetfilterQueue` package installed in a virtual environment.

```
cd firewall
# live mode only (Fedora package names):
sudo dnf install -y libnetfilter_queue-devel gcc python3-devel
python3 -m venv .venv && .venv/bin/pip install NetfilterQueue
```

6.2 Command-line tools

```
python3 pywall.py check                    # validate and list the rule set
python3 pywall.py simulate                 # the 11 scripted scenarios (73
packets)
python3 -m unittest discover -s tests -t . # the 43 unit tests
python3 pywall.py pcap demo/sample.pcap    # replay a capture through the
engine
```

6.3 The dashboard

1. Start it with simulated traffic (no root needed): `python3 pywall.py serve`.
2. Open http://127.0.0.1:8080.
3. On **Overview**, click a source, a stream, a stage column or a bin to follow that traffic. The panel on the right explains it and links to its packets.
4. In **Live traffic**, click any packet to see the stage-by-stage decision, the matched rule's conditions, every header field and the raw bytes.
5. In **Firewall rules**, select a rule to read it in plain language. You can then edit it, disable it, move it, or use **Test a packet** to see which rule a made-up packet would hit.
6. **Analytics** and **Security events** show the history of the last 1 to 15 minutes.

To slow everything down for a class:

1. Set **Speed** to ¼× or ⅒×.
2. Optionally choose **Show: Blocked only**.
3. Click **Watch a blocked packet** and step through its trip with **Next stage**.
4. Return with **Back to all traffic**, or press Esc.

Other traffic sources:

- Replay a capture: `python3 pywall.py serve --source pcap --pcap demo/sample.pcap --loop`
- Live kernel packets: `sudo .venv/bin/python pywall.py serve --source live ...` (see 6.5)

6.4 Live test on a private network

`sudo .venv/bin/python demo/live_demo.py` builds a virtual network: an "attacker" network namespace at 10.99.0.2 and 10.99.0.3, joined to the host at 10.99.0.1 by a veth pair. It starts PyWall on that interface only and runs:

- a web request
- blocked SSH and RDP

- outbound Telnet
- a spoofed ACK
- a ping burst
- a port scan
- a SYN flood

It then stops PyWall, removes the test network, and saves the session for the demo video. The real network is never touched.

6.5 Live demonstration on a real network

1. **Pre-flight check.** On the demo network, run:

   ```
   sudo .venv/bin/python pywall.py doctor
   ```

   It checks root, NetfilterQueue, iptables, the network interface, the detected LAN, the rule set and the dashboard port, and changes nothing.
2. **Rehearse safely.** Start a dry run, then open http://127.0.0.1:8080:

   ```
   sudo .venv/bin/python pywall.py serve --source live -i auto --lan auto --
   dry-run
   ```

   Every verdict is shown and nothing is dropped. The badge reads "Live, dry run: nothing dropped".
3. **Verify.** In a second terminal, without root, run `python3 demo/check_live.py`. It sends HTTPS, DNS, ping and Telnet traffic and confirms each verdict through the API.
4. **Enforce.** Press Ctrl+C, then start the same command without `--dry-run`. Run the checker again: Telnet must now be blocked, while browsing keeps working.
5. **Show attacks.** From another device on the same network, with the owner's agreement:
   - fast pings to the laptop hit the rate limit
   - `nmap <laptop-ip>` triggers port-scan detection and a 60-second ban
   - SSH from the LAN is allowed
6. **Stop.** Press Ctrl+C. PyWall removes its iptables hooks, and `sudo iptables -S | grep NFQUEUE` prints nothing.

The dashboard runs on 127.0.0.1, and the loopback interface is never filtered, so the browser showing it is unaffected. Every packet passes through Python, so heavy traffic such as video streaming should be avoided during the demonstration.

6.6 Demo videos

- `demo/pywall_demo.mp4` (101 s) shows rule validation, the attack simulation, a pcap replay and the unit tests.
- `demo/pywall_live_demo.mp4` (about 3 min) shows the live test on the private network.

Both are drawn as a Fedora terminal. Every command shown was really run and its output is real; `demo/record_demo.py` re-records them.

6.7 Public demo

A public, read-only copy of the dashboard runs at **https://pywall-demo.onrender.com/**. It uses the real engine on simulated traffic, because live filtering needs root and the kernel's netfilter on the protected machine, which a hosting platform does not provide. In read-only mode the server refuses rule changes and the dashboard hides its editing controls; the packet tester and everything else still work.

The service is defined in `render.yaml` in the repository (https://github.com/DE9856/Firewall-Implementation). Every push to the main branch redeploys it. On Render's free plan it sleeps when idle and takes about a minute to wake on the first visit.

7. Limitations and future work

- **IPv4 only.** IPv6 packets are not filtered. Support would need an `ip6tables` hook and IPv6 header parsing.
- **No payload inspection.** Attacks hidden inside allowed traffic (such as SQL injection over port 80) need an application-layer firewall or IDS.
- **User-space speed.** Every packet crosses into Python, which suits one host and a demonstration but not a busy gateway. A production version would compile its rules into nftables or eBPF.
- **Silent drops.** A dropped packet gets no TCP RST or ICMP unreachable reply. A REJECT action could be added.
- **Simple detectors.** Fixed thresholds can be evaded by slow or distributed scans.
- **Memory only.** Dashboard history lasts for the session, at most 15 minutes of counters and the latest 5,000 packets.
- **Cannot run on cloud platforms in live mode.** Filtering needs root and the kernel's netfilter on the protected machine. Hosts such as Render or Vercel could serve only a read-only demo with simulated traffic.

- **Local-only dashboard.** It has no login and is bound to 127.0.0.1; exposing it to a network would need authentication first.

8. Conclusion

PyWall meets all eight objectives:

- It captures real packets through NFQUEUE, parses their headers from raw bytes, and filters them with an ordered default-deny rule set.
- It tracks connections, drops spoofed and unsolicited traffic, rate-limits ping, and bans hosts that flood or scan.
- Its dashboard shows every decision as it happens and explains it field by field.
- It ran on a real Wi-Fi network with blocking on: Telnet was blocked, while browsing, DNS, ping and connections opened before start kept working.

All 43 unit tests and all 73 simulated packets pass.

The project shows how the theory of IP addressing, ports, the TCP handshake and ICMP turns into concrete security decisions. It also shows why a stateful filter protects far better than a stateless one.

References

1. J. Postel, RFC 791: Internet Protocol, 1981.
2. J. Postel, RFC 792: Internet Control Message Protocol, 1981.
3. W. Eddy, RFC 9293: Transmission Control Protocol, 2022.
4. J. Postel, RFC 768: User Datagram Protocol, 1980.
5. R. Braden, D. Borman, C. Partridge, RFC 1071: Computing the Internet Checksum, 1988.
6. The netfilter.org project, libnetfilter_queue documentation.
7. NetfilterQueue Python bindings, PyPI.
8. A. S. Tanenbaum, D. J. Wetherall, *Computer Networks*, 5th ed., Pearson, 2011, chapter 8 (Network Security: Firewalls).
9. J. F. Kurose, K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed., Pearson, 2021, section 8.9 (Operational Security: Firewalls and Intrusion Detection Systems).
10. R. Droms, RFC 2131: Dynamic Host Configuration Protocol, 1997.
11. WHATWG, HTML Living Standard: Server-sent events.
12. The netfilter.org project, Connection tracking (conntrack).